Mo Tu We Th Fr Sa Su

Date 12 / 10 / 24

CUDA

Global id formulae for x & y :

int global x = blockIdx.x * blockDim.x + threadIdx.x.

int globaly = blockIdx.y * blockDim.y + threadIdx.y.

2D Matrix from a 1D array:

(i) first, create array, d_x, then height & Width variables with values

Ex) int arr[12] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120};

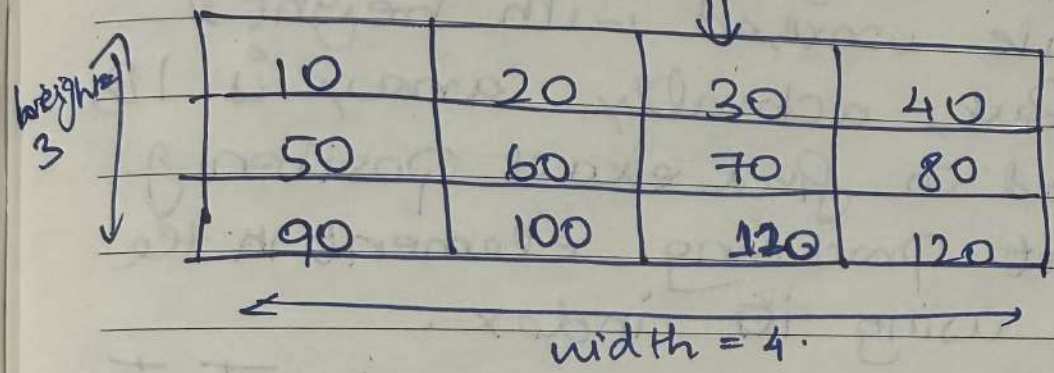
int height = 3;
int width = 4;

Now the array will be imagined like

Mo Tu We Th Fr Sa Su

Date / /

[10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]



(ii) Now, we will do usual things of creating Memory space, Allocating array here.

(iii) Now, inside global function, we need to have the global x & global y. int

```
__global__ void matrix (*array, int width, height){
int globalx = blockDim.x * blockIdx.x + threadIdx.x
int globaly = blockDim.y * blockIdx.y + threadIdx.y
if (globalx < width && globaly << height){
// parameters must not exceed what we have in array.
int index = globaly * width + globalx;
```

Mo Tu We Th Fr Sa Su

Date / /

This index is a new concept, here, we have imagined the array as 2D & have worked with height & width. But, actually, array is 1D, So, we need to find exact position of our current processing element in the array by using its index.

For ex)

(globalx, globaly)	10	20	30 → (we want to find its index in 1D array)	40
	(0,0)	(1,0)	(2,0)	(3,0)
	50	60	70	80
	(0,1)	(1,1)	(2,1)	(3,1)

It is, index = 0 (globaly) * 4 (width) + 2 (globalx)

= 2

In the 1D array it is stored in 2nd index.

Now, array[index] = array[index] * 2 ;

multiply the value with 2 @ index.

Mo Tu We Th Fr Sa Su

Date / /

While, in the main function we initiate the gpu process by:

matrix <<< dim3(2,2), dim3(2,2) >>> (d_x, width, height);

Now, we are launching 4 blocks each containing 4 threads => 4 x 4 = 16 threads in total

Finally, we print it.

Why 1D array represented as 2D image? Why not use 2D Array itself?

=> Absolutely, we can use a 2D array directly & it would simplify the code, but in computation, 1D array is better.

=> It is becoz, 1D array can store all the pixels as continuous buffer & can work naturally with many image processing library. Also, it is easier to allocate with cudaMalloc.

Each relationship nurtures a Strength or Weakness within you